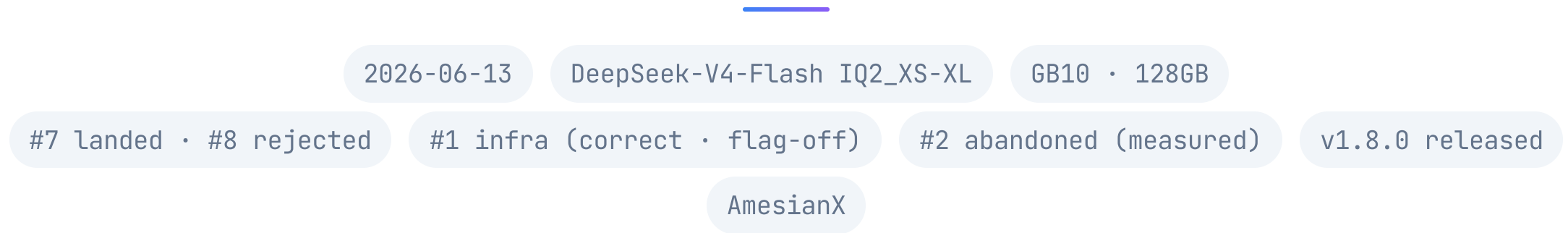


DeepSeek-V4 Optimization Sprint

verify-graph reuse infra · GPU-bound characterization · honest ceiling · v1.8.0 release



Session Overview

GOAL

DSV4 optimization sprint #1-14. Accelerate the MTP round via verify-graph reuse (+20-30% **expected**).

KEY FINDING

#1 is mathematically correct but **off the critical path**. Matmul ~52% near the LPDDR ceiling. Realistic envelope ~single-digit %.

OUTCOME

Measurement flipped the premise — **graph reuse is not the bottleneck**. DSV4 is at a memory-bound ceiling. v1.8.0 released.

DELIVERABLES

#7 committed, #1 infra (flag-off), 5-angle review, v1.8.0 release, 4 READMEs, `merge -s ours`.

Optimization Sprint #1-14 Results

measurement-driven

ITEM	STATUS	NOTES
#7 wq_b q8_0 small_k	landed	bit-identical (764/764), marginal — memory-bound
#8 iq2_xs occupancy	rejected	minBlocks=8 no-op (REG stayed 80, compiler ignored)
#1 verify-graph reuse	infra · flag-off	mathematically correct (proven), ~0 gain alone
#2 multi-slot cache	abandoned	measured: build is off critical path → useless
#3-#5 (MTP CPU)	held	mostly hidden behind the GPU (low ROI)
#6/#9-#14 (kernels)	held	matmul memory-bound / shuffle structural → ceiling

The sprint's real value was **"blocking a dead end"** — a high-risk framework change (#2) **pre-empted by measurement**.

#1 Debug: It Was Not a Bug resolved

flag-ON greedy diverged — SHA `aac34a0` vs baseline `a56fd72` , first split at token ~8 (abs pos ~21). **A topology change, yet the output differed.**

BISECTION EXPERIMENT	RESULT	CONCLUSION
flag-ON + NO_KREUSE (reuse off only)	still diverges	the BUILD, not reuse
+ RAWVIEW (force raw compressed-view)	baseline bit-identical	compressor · cache scatter · masks bit-perfect

Root cause: the **256-padded view** required for reuse only perturbs the flash-attn online-softmax **fp-accumulation order** → flips a near-tie argmax. Not a logic bug → must be perplexity-gated.

PIVOT: Graph Reuse Is Not the Bottleneck measured

CONFIG (SAME PROMPT)	GRAPHS REUSED	T/S
reuse fully OFF (rebuild every step)	0	19.80
reuse ON (single-slot)	16	19.83

Rebuilding every step costs ~0. **Graph build overlaps the async GPU compute** (the next graph builds on CPU while the GPU runs the current one) → **off the critical path**. Reuse only frees CPU work already hidden.

Result: **abandon #2**; #1 is correct but kept flag-off. The real bottleneck is the **target verify GPU pass (~25ms/round)**.

GPU Profile: DSV4 Is Memory-Bound

DSV4_KERNEL_PROF

GPU OP CLASS	SHARE	HEADROOM
MoE expert matvec (IQ2_XS, 256 experts) + dense attn projections (Q8_0)	~52%	near LPDDR ceiling — fusion saves launch only
data-shuffle (CPY/CONT/GET·SET_ROWS/CONCAT)	~19%	the real headroom, but structural (each op tiny)
flash-attention (D=512 compressed KV)	~2.3%	negligible — the compressed-KV design works

The matmuls read quantized weights from LPDDR every token → past that wall lies only **fewer bytes (a quality trade)**, not kernel fusion. Attention at 2.3% proves the **compressed-KV design works**.

Why the t/s Jitters

23 → 16 → 19

$t/s = (\text{tokens accepted per MTP round}) / (\text{round time})$. Round time $\approx$ constant ($\sim 25\text{ms}$, target verify GPU pass); tokens-per-round swing 1→3 with the **acceptance rate (96-100%)**.

- **Acceptance variance (dominant)** — predictable spans (lists, common phrasing) accept $\uparrow \rightarrow \sim 20$ t/s; novel content (numbers, names) $\rightarrow \sim 16$. Sentence-to-sentence swing = the **nature** of speculative decoding, not a bug.
- **Context-growth drift** — KV grows $\rightarrow$ round time creeps up. Step-jump at 256-multiple boundaries.
- **Metric conflation** — 23 (peak) / 16 (essay aggregate, incl. prompt-eval) / 19 (recent window) have different denominators.

Final Code Review

5 angles · 0 live bugs

CORRECTNESS

0 live bugs in the flag-off path. Findings all reduce to validated-pattern reuse / flag-off latent fragility.

HYGIENE

Diagnostics (NO_KREUSE/RAWVIEW) confirmed removed. `mmvq` `<=` is type-global (comment understates scope).

FIX APPLIED

Added the **fp-divergence hazard comment** at the `getenv` site (the warning lived only in `.md`).

RESIDUAL DEBT

Load-bearing invariants undocumented (verify-pos contiguity, `attn==index n_comp`) — assert them if #1 is ever enabled.

Git Work + v1.8.0 Release

- **FF-merge main (27 commits) + push origin** — full DSV4 port + session work, no merge commit, clean.
- **Delete feat/deepseek4** — local + remote (after confirming `main..branch = 0`).
- **Create v1.8.0 release** — moved tag to current main + GitHub release (Latest). The workflow clobbers binaries onto the latest release.
- **Upstream fringe** `merge -s ours` — 8 commits (we're ahead, 2287 vs 1392 lines) recorded as merged with 0 lines of code → clears the "behind by 8" warning.
- **4 README updates** (root+turboquant × EN+KO) — production perf + memory-bound characterization.
- **CI hygiene** — on change, all-stop (cancel) → run the final build fresh. `restore.sh` canonical check = no-op.

Missteps

retrospective

- **Over-framed the PIVOT as "abandon the sprint"** — measurement only killed #2, yet I reported as if folding everything. User reminded me of the "apply all 14" commitment; corrected.
- **Zombie monitor preserving the old build** — tried to keep the a4ab90b6c build alive. A CI-hygiene violation → fixed with all-stop + fresh, saved to memory.
- **#1 add/add 700-line debugging** — static analysis couldn't catch it; only the NO_KREUSE→RAWVIEW runtime bisection pinned it to fp-order. Should have isolated sooner.
- **Release-vs-tag confusion risk** — the workflow targets the latest **release**, but no v1.8.0 release existed → binaries were one step from landing on v1.7.0. Caught beforehand.
- **fringe merge add/add conflict** — almost regressed our 2287 lines down to 1392. User's warning + a dry-run blocked it just in time; selective port (= nothing to port).

Key Lessons

- **Measure before implementing** — pre-empted the high-risk #2 change with data. Blocked the "touched it but it didn't go up" trap.
- **Graph build overlaps async GPU = off critical path** — CPU-side optimizations (#1-#5) can't lift a GPU-bound decode.
- **Topology change $\neq$ bit-identical** — a padded view changes fp-accumulation order. Even "math unchanged" makes bit-identity too strict $\rightarrow$ judge by perplexity.
- **CI hygiene: all-stop $\rightarrow$ run the final build fresh** — never preserve in-flight builds. Prevents zombies / queue pollution.
- `merge -s ours` — a superior fork reconciles upstream divergence with 0 lines. Only real merges need the workflow-restore.
- **Memory-bound $\rightarrow$ only quant/expert count is a lever** — matmul 52% is the LPDDR wall; kernel fusion saves launch overhead only.

Next Steps

real levers only

#6 GATE+UP FUSION (GGUF REPACK)

Offline-concat iq2_xs gate+up → loader auto-picks it up. ~2-3% (memory-bound, launch only). 80GB file.

#11 COMPRESSOR SHUFFLE FUSION

Recover half of the 18.6% shuffle. High-risk, single-digit %. Custom fused kernel.

QUALITY TRADE-OFF

Fewer experts / lower quant. Big t/s gain possible but model-quality cost. User's call.

ASSERT ON #1 ENABLE

Pin the verify-pos contiguity / attn==index n_comp invariants with asserts + a perplexity gate.

DRAFT SAMPLER

~6.4ms/round (129k vocab) is high for greedy — inspect the CPU/sampler path.

ACCEPT THE CEILING

DSV4 is at its practical compute ceiling on GB10. Realistic envelope single-digit %.